

İçindekiler

Bugünkü Dönütler — 2026-09-04

1. "Addy'nin 25 skill'inden sadece 5'ini mi çağırıyoruz?"
2. "gstack ve superpowers'tan tam olarak ne aldık?"
3. "Repository ne demek? Bizde neye denk geliyor?"
4. "Contract ne demek?"
5. "Test uzmanına ihtiyacım var mı? TestSprite kullanmalı mıyım?"
6. "Docker'daki PostgreSQL'e bakıp tabloları ve ilişkileri görebilir miyim?"
7. "Bolt / Lovable / Base44'ün güçlü yanlarını da barındırsın"
8. İşyeri notları hakkında

Bugün kite giren değişikliklerin özeti (v1.76.0)

Bugünkü Dönütler — 2026-09-04

Bariş'in gönderdiği maddelere verilen cevaplar ve kite yansıyanlar. Kit sürümü bugün **1.75.0** → **1.76.0**.

1. "Addy'nin 25 skill'inden sadece 5'ini mi çağırıyoruz?"

Belgede hata vardı, ölçtüm: 5 değil 7 çağırılıyordu. Bugün 3 tane daha eklendi, toplam 10 oldu. **KIT-NE-YAPIYOR.md** düzeltildi.

Çağrılanlar

Skill	Nerede
<code>interview-me</code>	7 yerde — PRD görüşmesi
<code>frontend-ui-engineering</code>	Arayüz yazarken
<code>security-and-hardening</code>	Güvenlik denetimi
<code>source-driven-development</code>	Resmi dokümandan doğrulama
<code>test-driven-development</code>	Özellik geliştirirken
<code>code-review-and-quality</code>	Merge öncesi
<code>browser-testing-with-devtools</code>	Tarayıcı doğrulaması
<code>debugging-and-error-recovery</code>	Bugün eklendi
<code>incremental-implementation</code>	Bugün eklendi
<code>doubt-driven-development</code>	Bugün eklendi

Kalan 15 neden çağırılmıyor

Çoğunun kitede **kendi Türkçe karşılığı var**; ikisini birden çağırmak çelişki üretir.

Addy'nin skill'i	Kitteki karşılığı
<code>git-workflow-and-versioning</code>	<code>08-git-workflow.md</code>
<code>ci-cd-and-automation</code>	<code>09-ci-cd-deploy.md</code>
<code>api-and-interface-design</code>	<code>03-api-guidelines.md</code>
<code>observability-and-instrumentation</code>	<code>12-operations-and-scaling.md</code>
<code>documentation-and-adrs</code>	ADR şablonu + <code>teknoloji-ve-plan.md</code>
<code>spec-driven-development</code>	PRD akışı (Adım 3)
<code>planning-and-task-breakdown</code>	Roadmap + altı gözle denetim (Adım 4)
<code>shipping-and-launch</code>	Adım 6a/6b
<code>performance-optimization</code>	<code>07</code> performans bütçesi + <code>12</code>

Bugün eklenen üçünün gerekçesi

- **debugging-and-error-recovery** — kitte "üç başarısız denemede dur" kuralı vardı ama **nasıl** hata ayıklanacağı yazılı değildi
- **incremental-implementation** — "adım adım" deniyordu, yöntemi yoktu
- **doubt-driven-development** — kamu hizmeti ve geri alınamaz işlemde kararı taze bağlamla çapraz sorgular

2. "gstack ve superpowers'tan tam olarak ne aldık?"

Nereden	Ne alındı	Nereye yazıldı
gstack /design-consultation	Tasarım kararının nasıl verileceği: referansı incele → iki yön öner → taslak göster	SKILL.md Adım 3
gstack /design-review	AI slop yasak tablosu	07-ui-design-system.md
gstack /plan-*review	Roadmap'ın altı gözle denetimi	SKILL.md Adım 4 kapanışı
gstack /office-hours	Ürün değeri sorgulaması ("yapılmalı mı")	SKILL.md Adım 3
superpowers systematic-debugging	Üçüncü başarısız düzeltmede dur, mimariyi sorgula	11-agent-workflow.md

Alınmayan: superpowers'ın "insana sorma, devam et" felsefesi — CLAUDE.md kapı 2 ("plan sun, onayımı bekle") ile doğrudan çelişiyor.

3. "Repository ne demek? Bizde neye denk geliyor?"

Bir araç değil, **senin yazdığın kod katmanı**. Kural: Prisma çağrıları **sadece** o katmanda olur.

```
UI (React)          → "randevuları göster" der, nasıl geldiğini bilmez
↓
API (route handler) → Zod ile doğrular, yetki bakar, HTTP çevirir
↓
Servis (iş mantığı)  → "aynı gün ikinci randevu alınamaz" burada
↓
Repository (Prisma)  → SADECE veri erişimi: prisma.randevu.findMany(...)
↓
PostgreSQL
```

Projede karşılığı: **src/server/repositories/randevu.repository.ts** gibi bir dosya. İçinde iş kuralı **yok**. **Prisma** araç, **repository** o aracı kullanan katman.

Neden ayrı: yarın Prisma'dan başkasına geçersen sadece o katman değişir; ve iş kuralını veritabanına hiç gitmeden test edebilirsin.

4. "Contract ne demek?"

Frontend ile backend'in "bu API ne alır, ne döner" konusunda anlaşması. Stack'te `packages/contracts` klasöründe **Zod şeması** olarak duruyor — tek yerde tanımlı, iki taraf da aynısını kullanıyor.

Somut faydası: API'de `basvuruTarihi` alanını `tarih` diye değiştirirsen frontend **derlenmez**. Hata sen fark etmeden canlıya gidemez. Sözleşme olmasa hata çalışma anında, kullanıcının ekranında çıkardı.

5. "Test uzmanına ihtiyacım var mı? TestSprite kullanmalı mıyım?"

Hayır — ama bir test uzmanının kafasındaki listeye ihtiyacın var.

TestSprite senin durumunda **gereksiz**: zaten chrome-devtools MCP, TDD, axe (erişilebilirlik) ve Lighthouse (performans) var. TestSprite bunların üstüne bağımsız bir QA katmanı ekler — tek kişilik ekipte o katman zaten sensin. Ek araç ek bakım demektir.

ChatGPT'nin "önce kendin manuel test et" demesi doğrudu ama eksikti: **neye bakacağını** söylemiyordu.

Kite yazılan: BEŞ GÖZLE DOĞRULAMA (`06-testing.md`)

Bir özellik bitince sırayla:

#	Göz	Ajan ne yapar	Sen gözle neye bakarsın
1	Backend	Mutlu yol, hata yolları, yetkisiz erişim, sınır değerler (0, 1, çok uzun metin, Türkçe karakter), eşzamanlılık	— (ajan kanıt sunar)
2	Veri	Kayıt gerçekten yazıldı mı — <code>prisma studio</code> ile bakılır, yetim kayıt var mı	Studio ekranında kendi kaydını gör
3	Frontend	375/768/1440px · açık+koyu tema · konsol ve ağ hatası · dört ekran durumu	Ekran görüntüleri: metin taşıyor mu, hizalar tutuyor mu
4	Tasarım/UX	Tasarım ADR'sine uygun mu, AI slop var mı, her eylem 100ms içinde karşılık veriyor mu	Asıl senin katmanın: "çalışıyor ama saçma" dediğin yer
5	Güvenlik + işletme	Yetki ve sahiplik, girdi doğrulama, hata mesajı sızıntısı, N+1 sorgu, performans bütçesi	— (ajan kanıt sunar)

Artı iki zorunluluk

Etki alanı — her özellikte üç soru yazılı cevaplanır:

1. Hangi **başka ekranları** etkiliyor?
2. Hangi **başka API uçlarını** etkiliyor?
3. Veri modeli değiştiyse hangi **eski kayıtlar** etkilendi?

🚫 "Sadece şu dosyaya dokundum" cevap değildir. Etkilenen yerlerden **en az biri fiilen açılıp kontrol edilir**.

Öğretme zorunluluğu — ajan ne kontrol ettiğini **ve neden o kontrolü yaptığını** anlatır:

"Aynı anda iki randevu denedim, çünkü iki kişi aynı saniyede tıklarsa çift kayıt oluşabilir — benzersiz index bunu veritabanı seviyesinde engelliyor."

❌ "Test geçti" tek başına rapor değildir. Neyin test edildiği söylenmezse neyin test **edilmediği** bilinemez.

Yeni terimler `ogrendiklerim.md` → "Artık biliyorum" listesine ekleniyor; ajan sonraki oturumda o terimi baştan anlatmıyor.

6. "Docker'daki PostgreSQL'e bakıp tabloları ve ilişkileri görebilir miyim?"

Evet, TypeORM'a gerek yok — Prisma'nın karşılığı daha iyi.

```
npx prisma studio      # tarayıcıda localhost:5555
```

Excel tablosu gibi açılır; ilişki alanına tıklayınca bağlı kayda gidersin. Docker'daki Postgres'e `DATABASE_URL` üzerinden bağlanır, ek kurulum yok.

ER diyagramı Prisma'da hazır gelmiyor ama `prisma-erd-generator` ile `schema.prisma` 'dan **otomatik** üretiliyor — şema değişince diyagram da güncelleniyor.

❌ **Elle çizilmez.** Elle çizilen diyagram ilk şema değişikliğinde yalan söyler ve kimse fark etmez.

❌ **Üretim veritabanına Studio ile bağlanılmaz.** Yanlışlıkla kayıt silmek tek tıklıktır.

Senin sorduğun ayrım — doğru soruydu

Proje	Gerekli mi
Backend'i biz yazıyoruz	✅ Evet — veri modelini görmeden doğru API tasarlanmaz
API tasarlıyoruz	✅ Evet
Yalnızca hazır API tüketiliyor	❌ Hayır — bize lazım olan API sözleşmesi

7. "Bolt / Lovable / Base44'ün güçlü yanlarını da barındırsın"

Üçünün gerçek gücünü ayıkladım; beşinden üçü alınabilirdi.

Güçlü yanı	Durum
Anında çalışan şeyi görmek	✅ Adım 5'e madde 6 eklendi — iskelet biter bitmez <code>npm run dev</code> + ekran görüntüsü + Prisma Studio, hesap istemeden önce
Varsayılan olarak iyi görünen çıktı	✅ Zaten vardı — tasarım yönü ADR'si + AI slop yasakları
Kapalı hata döngüsü	✅ <code>debugging-and-error-recovery</code> + üç deneme kuralı
Sıfır kurulum sürtünmesi	⚠️ Kısmen — kit bilerek soru soruyor (PRD), ama artık çalışan şeyi görmeden hesap açtırmıyor
Tek tıkla yayın	❌ Mümkün değil — hesap açmak kimlik doğrulaması ister, ajan giremez. Kısıt, kusur değil

Söylenmesi gereken gerilim

Bolt ve Lovable öğretmez, gizler. Senin istediğin ikisi birden: otomasyon **ve** her adımda öğrenmek. Otomasyon arttıkça öğrenme azalır — bu doğal bir gerilim.

Kitin farkı burada: Lovable "*işte siteniz*" der, senin kitin "*şunu şu yüzden şöyle yaptık, bak*" der. Sendeki asıl değer üretilen kod değil, **kodu anlıyor olman**. Bu yüzden öğretme tarafı feda edilmedi.

8. İşyeri notları hakkında

O blok kit malzemesi değil, kendi durumunu anlatan notlar — kite yansıtmadım. Ama içinde kitle doğrudan ilgili bir tespit var:

"AI'ın yazdığı kodu senior yazılımcılar bile okuyup net anlayamıyor... bir yer bozulsa hatayı AI'a atıp çözdürtmek dışında yapabilecekleri bir şey yok."

Bu, kitinin zaten savaştığı şey: `02-coding-standards.md` → "*Kod, okuyamayan biri için de anlaşılır olur*", `CLAUDE.md` 'nin öğretme kuralları, `ogrendiklerim.md`. Kitin ürettiği fark tam olarak bu — **sende kod sahipliği var**, çünkü kit her adımda *neden* öyle yapıldığını yazmaya zorluyor.

Bugün kite giren değişikliklerin özeti (v1.76.0)

Ne	Nerede
Beş gözle doğrulama + etki alanı + öğretme zorunluluğu	06-testing.md
Definition of Done'a dört yeni madde	10-definition-of-done.md
Prisma Studio + ER diyagramı + hangi projede gerekli	04-database.md
Üç yeni addy skill'i (gerekçeleriyle)	SKILL.md Adım 0
"Çalışan şeyi göster" adımı	SKILL.md Adım 5, madde 6
Rol tablosuna "Test / doğrulama" satırı	CLAUDE.md
5 → 8 skill düzeltmesi	docs/KIT-NE-YAPIYOR.md

Üretilen belgeler: [KIT-NE-YAPIYOR.md](#) (referans) · [KIT-REHBER.md](#) (her terim açıklamalı) · ikisinin de PDF'i.